CPSC 370 · AGENTIC ARTIFICIAL INTELLIGENCE · FALL 2026

Assignment 1
The Contract Net Tournament

Build an autonomous agent that bids on real computational work, then run it against the whole class in a live tournament.

SUBMIT	WORK IN	TOURNAMENT	EVERYTHING DUE
Two files, no zip	**Teams of 1 or 2**	**In class, Thu Sep 10**	_____

1 What you're building

You will write a **contractor agent**. The **manager** runs on the course server. It announces work that needs doing, your agent decides whether it wants the job and what to charge, and if it wins, your laptop performs the computation and returns the answer.

The work is real. When you win a contract, your machine runs a Monte Carlo simulation, or counts primes, or grinds through a proof-of-work search, and the manager checks your answer against a key you don't have. So bidding well means knowing something true about your own machine and about the job in front of you.

Here's the question your agent has to answer, in about a second, over and over, without enough information:

> Given what I know about my own computational capacity, what I've already committed to, and this particular job, should I bid at all? And if so, what should I charge?

Bid too high and you never win anything. Bid too low and you win everything at a loss. Promise a delivery time you can't hit and you get fined. Refuse everything and you end with nothing. There's no dominant strategy here, because the right answer depends on what your classmates do, and they're adapting too.

2 Background: the Contract Net Protocol

Where it came from

In 1980 Reid G. Smith published *The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver* in IEEE Transactions on Computers. He was working on distributed sensor networks and had a problem that should sound familiar: a bunch of independent nodes, each with different capabilities and different local information, needed to divide up work, and no single node knew enough to make the assignment centrally.

His solution was to stop trying. Instead of a scheduler that decides who does what, he borrowed the structure of commercial contracting. Work gets *announced*. Interested parties *bid*. The announcer *awards*. The winner *performs and reports*.

The insight is that the node best suited to a task is usually the node in the best position to know that. It's the only one that knows its own load, its own capabilities, and its own local data. A central scheduler would have to be told all of that continuously by everybody. A market just asks.

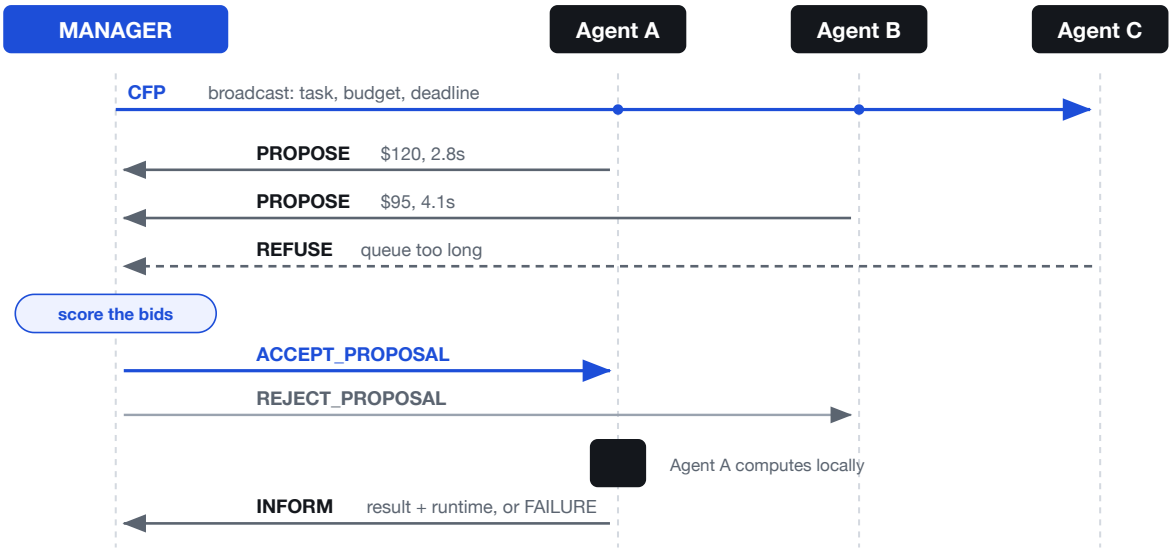
Two roles

There are two roles, and they're roles rather than fixed identities. The **manager** has a task it wants done: it announces the task, evaluates bids, awards a contract, and validates the result. A **contractor** evaluates announcements against its own situation and decides whether to offer to do the work.

In a full Contract Net, any agent plays either role at different moments, and a contractor that wins a big job can decompose it and become the manager of several sub-contracts. This assignment uses the simple version: the course server is always the manager, and you are always a contractor.

The conversation

A single contract is a fixed sequence of messages. This is the entire protocol.



One contract, start to finish. The manager broadcasts a call for proposals; contractors bid or refuse; the manager scores the bids and tells everyone who won; the winner computes and reports back.

Four things in that diagram are worth pausing on.

Refusal is a real message. `REFUSE` isn't an error or a timeout. It's a legitimate, expected reply that carries information. An agent that knows it shouldn't take a job says so.

The manager cannot compel anybody. If every contractor refuses, the task does not get done, and the manager's only recourse is to announce it again, perhaps on better terms. Agent autonomy is preserved by construction, not by politeness.

Losers get told they lost. `REJECT_PROPOSAL` matters because a contractor that bid has tentatively set aside capacity for that job. It needs to know it can release it.

Failure is part of the protocol, not an exception to it. `FAILURE` and the manager's timeout are ordinary paths through the state machine. Undelivered work goes back on the market and gets re-announced.

FIPA and the message names

In 2002 the Foundation for Intelligent Physical Agents standardized this as the FIPA Contract Net Interaction Protocol and fixed the performative names: `cfp`, `propose`, `refuse`, `accept-proposal`, `reject-proposal`, `inform`, and `failure`. The message names in this assignment are those names in capitals. When you write `PROPOSE` in your agent, you're speaking a twenty-year-old standard vocabulary that anybody who has built on JADE would recognize.

What it buys you, and what it costs

STRENGTHS	WEAKNESSES (YOU WILL FEEL ALL OF THESE)
No central scheduler, so no single bottleneck and no single point of failure. Heterogeneous agents are handled naturally, since nobody maintains a registry of who can do what. Load balances opportunistically, because a busy agent bids worse or refuses. Private information stays private: a bid reveals a price, not a capability profile. Failure recovery is just re-announcement.	Communication cost grows with the number of contractors, and every task costs a full round of messages. No guarantee of a globally optimal allocation, since it's greedy contract by contract. It assumes bidders are honest about what they can do, which makes it vulnerable to strategic misreporting. And the *eager bidder* problem is real: an agent that bids on everything can win more than it can deliver, because bids get evaluated before commitments settle.

That last one isn't a bug in your implementation. It's a genuine open problem in the protocol, and dealing with it is a big part of what makes this assignment interesting.

Why this is in an agentic AI course

Contract Net is forty-five years old and more relevant now than when it was written. The current generation of LLM agent frameworks has rediscovered exactly this shape: an orchestrator decomposes a goal and farms subtasks out to specialized workers, which report results back. Cloud and grid schedulers use auction mechanisms descended from it. Multi-robot task allocation is largely Contract Net with better math.

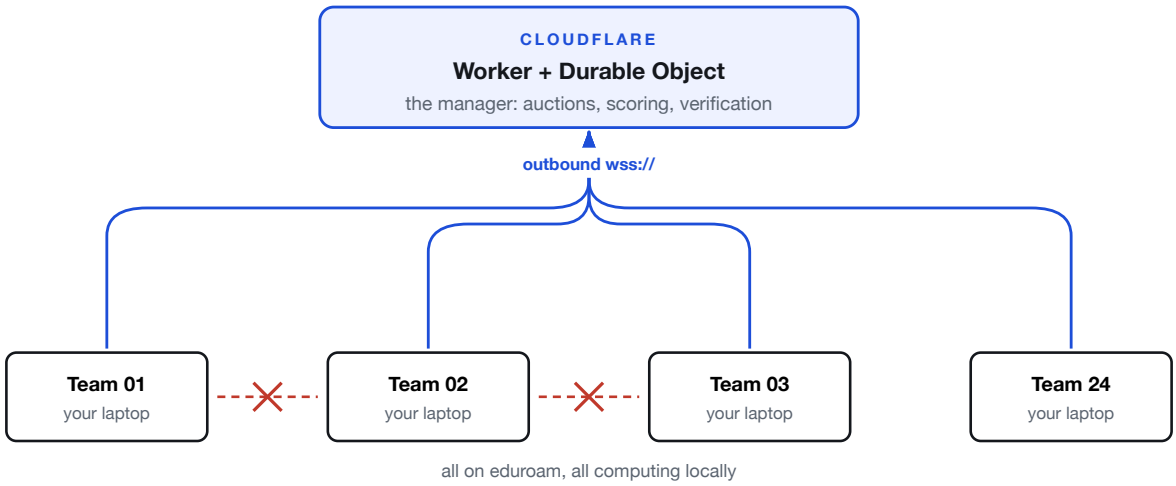
What hasn't changed is the hard part, and it's the part you're implementing: an agent deciding, from its own local view, whether to commit to something. Every question you hit this week is a question any agent taking actions in the world has to answer.

3 Learning objectives

By the end of this assignment you should be able to:

1. **Implement a FIPA-style Contract Net contractor** against a real asynchronous message-passing protocol, including the failure cases.
2. **Reason about self-knowledge in agents.** Calibrate an agent's model of its own capability and use that model to make commitments.
3. **Price risk.** Some tasks have predictable cost. At least one doesn't. Your bid has to reflect the difference.
4. **Handle commitment under constraint**, meaning deadlines, queues, and the consequences of over-promising.
5. **Analyze a multi-agent system empirically**, using data from a live run against real competitors rather than a simulation you control.

4 How it's wired, and why



Everything is an outbound connection. Your laptop never accepts an inbound connection and never talks to another student's laptop, which is good, because campus Wi-Fi won't let it.

That shape isn't an accident. University Wi-Fi normally isolates clients from each other, so you and the person sitting next to you can both reach the internet while being completely unable to reach each other. A peer-to-peer version of this exercise would spend the whole class period failing to connect. Routing everything through one publicly reachable coordinator sidesteps that entirely, and it also happens to be how most real multi-agent systems get deployed.

The manager coordinates. It does not compute. All of the actual work happens on your machine, which is what makes this a genuine distributed system instead of a simulation.

5 Getting set up

Download `contract-net.zip` from Canvas and unzip it wherever you keep coursework. You'll get a `student/` folder, which is everything you need. There's one dependency.

```
cd contract-net/student
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Check that the reference tasks run on your machine:

```
python verify.py
```

That does two things: it confirms the five task implementations produce the known-good answers, and it times them. Those timings are your machine's fingerprint, and they'll be different from everybody else's. That difference is the entire economy.

Now connect to the practice room:

```
python my_contractor.py --name YourTeamName --url PRACTICE_URL --token CLASS_TOKEN
```

> **URLs and the class token are in the Canvas announcement.**
>
> Choose a team name now and use that same name for the whole assignment, in the practice room and in the tournament. Your results are matched to your submission by that name, and it is how you find yourself on the leaderboard. Changing it partway through breaks the link between your practice history and your tournament run.
>
> Keep it appropriate. Your team name appears on a projector in front of the class and stays in the published results, so treat it the way you would treat anything else you hand in with your name on it. Anything that does not clear that bar will be renamed, and you will bid the rest of the tournament as `Team_Anonymous`.

The practice room runs continuously and isn't competitive. It issues sample jobs on a loop, forever, so you always have live traffic to develop against. Nothing there is scored and nothing there counts.

Two dashboards

DEVELOPMENT DASHBOARD, AT `PRACTICE_URL/DEV`

This is the one you'll live in. Pick your team name from the focus dropdown and it shows you only your agent's traffic: the current call for proposals and its exact parameters, every bid on it including the ones that got thrown out *with the reason*, which connected agents said nothing at all, and the full protocol conversation message by message in both directions.

If your agent isn't winning anything, this page tells you why. The three most common causes are all visible here: a bid over budget, an estimate past the deadline, or no response at all because `on_cfp` threw an

exception.

TOURNAMENT DASHBOARD, AT `TOURNAMENT_URL/`

Standings, revenue, cost, fines, profit, and a live event feed. This goes on the projector on tournament day.

6 What you actually write

Open `student/my_contractor.py`. Everything else in `student/` is plumbing you shouldn't need to touch.

You implement one method:

```
class MyContractor(Contractor):
    def on_cfp(self, task: Task) -> Bid | None:
        """Return a Bid to compete for this contract, or None to refuse."""
```

The starter version is deliberately mediocre. It applies a flat markup and ignores everything interesting. It will connect, bid, win some work, and lose money. Start by watching it do exactly that.

WHAT YOU HAVE TO WORK WITH

ON `SELF`	MEANING
`self.estimate(task)`	predicted seconds for this task on this machine
`self.queue_seconds`	seconds of work you're already committed to
`self.rules`	the manager's published scoring rules
`self.history`	every settlement you've received so far
`self.profit`	your running profit
`self.rates`	your calibration, in work units per second, per task type

ON `TASK`	MEANING
`task.task_type`	which of the five kinds of job this is
`task.params`	the parameters of this specific instance
`task.budget`	the maximum the manager will pay; a higher bid is void
`task.deadline_s`	seconds you get, counted from the moment you win
`task.work`	a proportional estimate of how much work this is
`task.attempt`	1 normally; higher means somebody already failed this job

There are optional hooks too, if you want them: `on_registered`, `on_reject`, `on_settled`, and `on_bid_invalid`.

7 The rules of the economy

Winning the auction

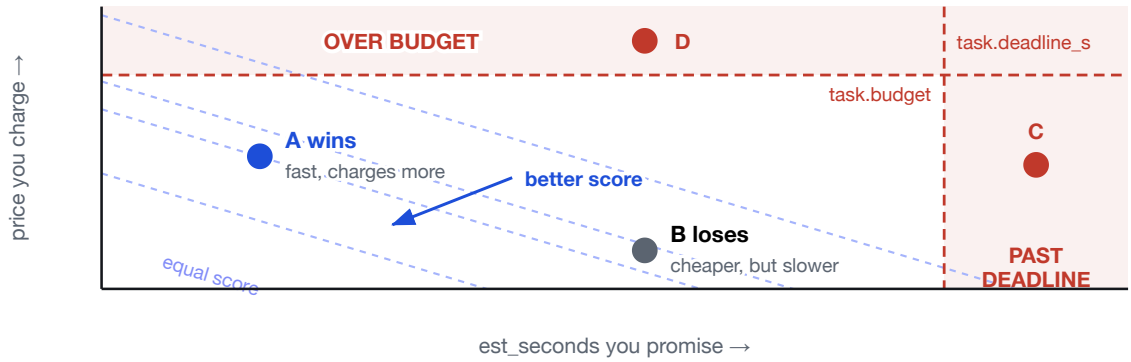
Bids get scored and the **lowest score wins**. Under the default policy:

```
score = price + time_weight × est_seconds
```

`time_weight` is published to your agent in `self.rules`. Do not hardcode it. It may change between the practice room and the tournament, and any change will be announced.

A bid is thrown out before scoring if `price > task.budget` or if `est_seconds > task.deadline_s`. Your agent gets told immediately when this happens, and it shows up on the development dashboard.

Ties break by estimated time, then by which bid arrived first, then alphabetically. The auction is fully deterministic, so the numbers behind any result are all on the dashboard.



The bid space. Anything above the budget line or right of the deadline line gets discarded before scoring, so C and D never compete. Among what is left, the winner is whichever bid sits furthest down the dashed iso-score lines. A is more expensive than B and still wins, because `time_weight` makes B's extra seconds cost more than it saved on price. That is why calibrating your machine matters: being genuinely fast lets you charge for it.

Getting paid

```
cost    = actual_runtime × cost_rate    (always charged, the machine ran)
revenue = price                       if correct and on time
        = price × late_credit          if correct but late
        = 0                           otherwise
penalty = budget × penalty_rate       on anything but a clean delivery
profit  = revenue − cost − penalty
```

Four consequences worth internalizing.

1. **Refusing is free.** Losing an auction costs you nothing at all. Winning one you then botch costs you real money. Silence is a legitimate strategy.

2. **Your compute is billed whether or not you deliver.** A job you start, run for four seconds, and then fail is a four-second loss. The one exception is a timeout: the manager never learns how long you actually worked, so it bills the runtime you promised rather than the whole window.
3. **Breach damages come off the budget, not off your bid.** This is deliberate and you should think about why. If the penalty were a share of your own bid, the winning strategy would be to bid a penny on everything, win by undercutting, never deliver, and pay a penny each time. Tying damages to the value of the work means your downside is fixed, is printed on the CFP before you bid, and is the same for everybody.
4. **The deadline runs on the manager's clock**, from award to the arrival of your result, so network latency counts against your delivery time. Deadlines have enough slack that this is a small effect, but it is not zero, and it is the same for everyone.

Failure and re-contracting

If you win a job and do not deliver in time, the manager fines you, takes the contract back, and re-offers it to the room. You will see these as CFPs with `task.attempt > 1`. Somebody else's failure is your opportunity. This is also how the demo at the end of the tournament works, so expect it.

8 The five task types

All five are deterministic and return an exact integer, so verification is exact. There's no tolerance and no partial credit on a wrong answer.

TASK	WORK SCALES WITH	NOTES
`monte_carlo_pi`	`samples`	Seeded points in the unit circle. Very predictable.
`prime_count`	`(hi − lo) · √n`	Trial division over a range. Very predictable.
`hash_search`	`2^32 / threshold`, *on average*	Smallest nonce whose SHA-256 digest falls below a target. Read the warning below.
`sort_checksum`	`n log n`	Sort seeded integers, fold into a checksum. Predictable.
`matmul_mod`	`n³`	Integer matrix multiply mod a prime. Predictable.

> **Read that third row again.** `hash_search` is a proof-of-work search, sized the way Bitcoin sizes one: each attempt succeeds with probability `threshold / 2^32`, so the *expected* number of attempts is `2^32 / threshold`. Any individual instance is a geometric random variable. It can easily take two or three times the expectation, and sometimes it finishes almost immediately. You cannot predict a specific instance, only its distribution.
>
> The manager knows exactly how long each instance takes, because every task was solved in advance, and each task's budget and deadline are set from that measurement. You do not get to see it. Pricing this uncertainty is the sharpest single decision in the assignment. In testing it was the difference between first place and a net loss.

9 What to hand in

Upload two files to Canvas. **Do not zip them, and do not submit anything else.**

FILE	WHAT IT IS
`my_contractor.py`	Your agent, as you ran it in the tournament.
One PDF	Your strategy and your analysis together. Two pages maximum.

Both files are due _____. There is no earlier deadline for the code. You run your own agent on your own machine during the tournament, so nothing has to be submitted in advance. Upload both files together once you have written up the results.

> **If you worked as a pair, both of you submit.** Each of you uploads the same two files to your own Canvas submission. Do not submit different versions, and do not split the work across the two submissions. Put both names and your team name on the first page of the PDF so it can be matched to your agent in the tournament results.
>
> A missing upload is a missing assignment, whatever your partner submitted.

1. `my_contractor.py`

Your agent. It must run with only `--name`, `--url`, and `--token` on the command line, and it must be the version you actually competed with. Do not change anything else in `contractnet/`: the manager grades against the reference implementations, so a modified copy returns wrong answers and is fined.

2. One PDF, covering both halves

Write it up as a single document. Write part one before the tournament, while you still do not know how it turns out, and part two afterwards. Reconstructing your reasoning after seeing the standings is not the same exercise, and it reads that way.

Two pages maximum, for both parts together. That is a limit, not a target, and it is the hardest part of the write-up. Everything below must be covered, so you will have to decide what actually mattered and cut the rest. Do not pad it, and do not paste in your source code.

PART ONE: YOUR STRATEGY

Cover:

- **Your bidding function.** What does your agent charge, and why? Give the reasoning, not just the code.
- **Estimation.** How do you predict your own runtime, and how did you check that prediction against reality? Include evidence from practice runs.
- **Risk.** How do you handle `hash_search` specifically, and uncertainty generally? If you treat it the same as the other four, defend that.

- **Refusal.** When does your agent decline work, and why is that the right call?
- **What you tried that did not work.** An honest account of a strategy that lost money, and why it lost money, is worth more than a polished story about one that worked.

PART TWO: WHAT ACTUALLY HAPPENED

Written after the tournament, using the published results data.

- Where did you finish and why? Attribute your profit or loss to specific decisions on specific tasks.
- Pick one competitor who beat you and reconstruct what they did differently from the public bid record.
- What would you change if we ran it again tomorrow?

3. Show up

Be in class on Thursday with a working, connected agent, and leave it running for the whole tournament. You run your own agent on your own machine, so if you are not there, your agent does not compete.

10 Going further

Optional, and worth doing if you finish early.

Beat the reference implementation. Override `execute()` with something faster. It must return the identical integer: a mismatch is scored as a wrong answer and fined, in the tournament as well as in practice. `matmul_mod` with NumPy is the obvious target, and watch your integer overflow. Run `python verify.py`, which checks your version against the reference and reports your speedup, and put those numbers in your write-up.

A genuinely faster executor changes your whole economic position, because it lets you quote shorter times and still charge well. That interaction is worth writing about.

11 Ground rules

Encouraged: studying the public bid history, modeling your competitors, adapting your strategy mid-tournament based on what you observe, being strategically unpredictable, and refusing work you don't want.

Not allowed:

- Modifying anything in `contractnet/` other than `my_contractor.py`.
- Misreporting your runtime, or any other attempt to misrepresent what your agent did. The manager records both your self-reported runtime and its own measurement, and compares them.
- Trying to obtain the answer key, or to reach any endpoint under `/admin/`.
- Interfering with another team's agent or with the manager. Connecting under another team's name is impersonation and will be treated as an integrity violation, not as a clever hack.
- Coordinating bids with another team to manipulate the auction. Compete independently.

> If you find a real bug or exploit in the manager, and there may well be one, report it. Reported findings earn credit. Exploiting one quietly is an integrity violation.

On using AI

AI tools are allowed. So is talking to other teams about strategy.

The requirement is that you understand your own costing strategy. You must be able to explain, out loud and without notes, why your agent charges what it charges, what it does when its queue is full, and how you arrived at your treatment of `hash_search`. You may be asked to do so. If there is a line in your bidding logic you cannot account for, remove it.

Generated code you can defend is fine. Generated code you cannot explain will be bidding your money on Thursday against agents whose owners tuned theirs.

12 When things go wrong

SYMPTOM	WHAT'S USUALLY HAPPENING
Connects but never bids	Open the development dashboard, focus on your team, and find the CFP in the trace. If there's no `PROPOSE` or `REFUSE` after it, your `on_cfp` threw. The SDK catches that, logs it, and refuses on your behalf. Check your terminal.
Bids always thrown out	The dashboard shows the reason on the bid row, and your agent gets told directly. Almost always price over budget, or estimated time past the deadline.
Wins everything, loses money	You're underpricing. Compare `est_seconds` against the actual `runtime` in your settlement messages. If your estimates run systematically low, your calibration is optimistic and your markup isn't covering the gap.
Never wins anything	You're overpricing, or over-padding your time estimate. Remember that padding `est_seconds` to feel safe also makes your bid score worse.
Connection keeps dropping	Expected on campus Wi-Fi. The SDK reconnects with backoff and re-registers you. If you were mid-execution, the result still gets delivered when the socket comes back. Bids for auctions that closed while you were away are discarded rather than sent late.

13 The whole assignment, in short

Everything above in one list. If you only read one section, read this one, then go back for the parts you need.

Set up, once

1. Download `contract-net.zip` from Canvas and unzip it.
2. In `contract-net/student`, make a virtual environment and install the one dependency:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

3. Run `python verify.py`. All five checks must pass before you go any further.
4. Pick a team name. Use that same name every time, in practice and in the tournament, and keep it appropriate.

Build your agent

5. Connect to the practice room and watch the starter agent lose money:

```
python my_contractor.py --name YourTeamName --url PRACTICE_URL --token CLASS_TOKEN
```

6. Open `PRACTICE_URL/dev`, pick your team from the focus dropdown, and keep it open. It shows every message your agent sends and receives, and the reason any bid of yours was thrown out.
7. Edit `on_cfp` in `my_contractor.py`. That one method is the assignment. Decide whether to bid, and what to charge, using `self.estimate(task)`, `self.queue_seconds`, `self.rules`, and `self.history`. Appendix A is the full API.
8. Keep notes as you go on what you tried and what happened. You need them for the write-up, and they are much harder to reconstruct afterwards.
9. Do not edit anything in `contractnet/`. If you do, `verify.py` will fail and every answer you submit will be graded wrong.

Thursday, September 10

10. Write part one of your PDF, on your strategy, **before** the tournament. Reconstructing your reasoning afterwards reads exactly like what it is.
11. Come to class with your agent running and connected, and leave it running for the whole tournament. Nothing is submitted in advance.

After the tournament

12. Write part two from the published results: where you finished and why, one competitor who beat you and what they did differently, and what you would change.
13. Combine both parts into **one PDF, two pages maximum**, with both names and your team name on the first page.
14. Upload `my_contractor.py` and the PDF to Canvas as **two separate files**, not a zip. **If you worked as a pair, both of you upload both files** to your own submission.

> **The two things that most often go wrong.** Bids are thrown out for exceeding `task.budget` or `task.deadline_s`, and agents go quiet because `on_cfp` raised an exception. Both are visible on the development dashboard, and both are in section 12.

Good luck. Bring a charger.

A Appendix: the contractor API

Everything in this appendix lives in `contractnet/`, which you import but do not edit. Section 6 covers what you write. This is the full reference, with code that runs as printed.

A.1 A complete agent

This is a whole working contractor. It connects, calibrates, bids on everything it thinks it can finish, does the work, and reports back.

```
from contractnet import Bid, Contractor, Task

class MyContractor(Contractor):
    def on_cfp(self, task: Task) -> Bid | None:
        seconds = self.estimate(task)
        if seconds > task.deadline_s:
            return None
        return Bid(price=seconds * 2.0, est_seconds=seconds)

MyContractor(
    name="Team_07",
    url="wss://.../agent",
    token="the class token",
).run()
```

`run()` blocks until you interrupt it. It calibrates your machine first, which takes a second or two, then connects and stays connected, reconnecting on its own if the wifi drops.

A.2 Which method runs when

The manager drives your agent. You never call these; you implement the ones you care about and the SDK invokes them as messages arrive.

METHOD	FIRES WHEN	NOTES
`on_cfp(task)`	A task is announced	**Required.** Return a `Bid` or `None`. If it raises, the SDK logs it and refuses for you.
`execute(task)`	You won, and it is your turn	Runs on a worker thread, so blocking is fine. Must return an `int`.
`on_registered()`	The manager accepts you	Optional. `self.rules` is populated by now, not before.
`on_reject(task_id, winner, price)`	Someone else won	Optional. This is how you watch your competitors.
`on_settled(settlement)`	A contract of yours is graded	Optional. The settlement is already appended to `self.history`.
`on_bid_invalid(task_id, reason)`	Your bid was thrown out	Optional. Fires before the auction is scored.

> `on_cfp` runs on the event loop, so keep it fast. Do not compute anything real in there. You have a few seconds to answer a call for proposals, and anything slow in `on_cfp` delays every other message your agent is handling. Actual work belongs in `execute`.

A.3 What you know about the job

ON `TASK`	TYPE	MEANING
`task.task_id`	int	Identifier. Use it to match settlements to bids.
`task.task_type`	str	One of the five task names.
`task.params`	dict	The parameters of this instance, for example `{"n": 260, "mod": 1000003, "seed": 883211}`.
`task.budget`	float	Maximum the manager will pay. A higher bid is void.
`task.deadline_s`	float	Seconds allowed, from the moment you win.
`task.bid_window_ms`	int	How long you have to answer.
`task.attempt`	int	1 normally. Higher means somebody already failed this one.
`task.work`	float	Proportional size of the job, in abstract work units.

A.4 What you know about yourself

ON `SELF`	TYPE	MEANING
`self.estimate(task)`	float	Predicted seconds, from `task.work` divided by your measured rate.
`self.queue_seconds`	float	Seconds of work you are already committed to, counting down as the current job runs.
`self.rates`	dict	Work units per second, per task type. Your calibration.
`self.history`	list	Every `Settlement` you have received.
`self.profit`	float	Running total across `self.history`.
`self.rules`	Rules	The manager's live scoring configuration.

SELF.RULES

Read these instead of hardcoding numbers. They arrive at registration and may change between the practice room and the tournament.

FIELD	MEANING
`rules.award_policy`	`"best_value"`, `"lowest_price"`, or `"earliest_finish"`.
`rules.time_weight`	Currency per estimated second in the `best_value` score.
`rules.cost_rate`	What a second of your runtime costs you.
`rules.penalty_rate`	Breach damages as a share of `task.budget`.
`rules.late_credit`	Share of price still paid for correct but late work.
`rules.concurrency`	How many tasks can be in flight at once. Above 1, you can be awarded a second job while still running the first.

SETTLEMENT

What lands in `self.history`, and what `on_settled` receives.

```
Settlement(
    task_id=42,
    task_type="matmul_mod",
    verdict="correct",     # or wrong_answer, late, timeout, failure
    revenue=2.75,
    cost=1.09,
    penalty=0.0,
    profit=1.66,
    runtime=1.093,         # measured by the manager, and what you were billed
    est_seconds=1.1,       # what you promised when you bid
)
```

A.5 Learning from your own history

The single most useful thing in `self.history` is the gap between `est_seconds` and `runtime`. Your calibration is measured once at startup on small instances, and it will drift: thermal throttling, a browser eating your cores, a task type that scales differently than the work model assumes. This reads that gap back and corrects for it.

```
class MyContractor(Contractor):

    def bias(self, task_type: str) -> float:
        """How much longer jobs really take than predicted, as a ratio."""
        ratios = [
            s.runtime / s.est_seconds
            for s in self.history
            if s.task_type == task_type and s.est_seconds and s.runtime
        ]
        if not ratios:
            return 1.0
        return sum(ratios) / len(ratios)

    def on_cfp(self, task):
        seconds = self.estimate(task) * self.bias(task.task_type)
        finish = self.queue_seconds + seconds
        if finish > task.deadline_s:
            return None
        price = seconds * self.rules.cost_rate * 1.5
        if price > task.budget:
            return None
        return Bid(price=price, est_seconds=finish)
```

That is an illustration of the API, not a strategy. It corrects one thing and ignores the deadline margin, the auction score, and the cost of being wrong. Those are yours to work out.

A.6 Doing the work

`execute` receives the same `Task` you bid on and must return the same integer the reference implementation would. It runs on a worker thread, so blocking calls are fine and will not stall your connection.

```
def execute(self, task: Task) -> int:
    if task.task_type == "matmul_mod":
        return self.matmul_fast(task.params)
    return super().execute(task)      # everything else: use the reference
```

The trap is not the arithmetic, it is the inputs. Every task builds its data from a seeded `random.Random`, and the values depend on the exact sequence of calls. Generate the inputs the same way the reference does, then hand them to whatever fast library you like. Generation is cheap next to the work, so you lose almost nothing by keeping it in pure Python.

```
def matmul_fast(self, params) -> int:
    from random import Random
    import numpy as np

    rng = Random(int(params["seed"]))
    n, mod = int(params["n"]), int(params["mod"])
    a = np.array([[rng.randrange(mod) for _ in range(n)] for _ in range(n)],
                 dtype=np.int64)
    b = np.array([[rng.randrange(mod) for _ in range(n)] for _ in range(n)],
                 dtype=np.int64)
    return int(((a @ b) % mod).sum() % mod)
```

Watch the integer width. Entries are under a million, products are under 10^{12}, and a row of 300 sums to under 10^{15}, which fits in `int64` with room to spare. Use `float64`, which is what NumPy picks by default for some operations, and you lose exactness somewhere past 2^{53} and start silently returning wrong answers.

That version runs roughly 28 times faster than the reference at `n = 260` on a recent laptop. NumPy is not in `requirements.txt`, so `pip install numpy` if you want to try it. Run `python verify.py` afterwards: it runs both implementations on fixed inputs, compares them, and reports the speedup.

A.7 Running it

```
python my_contractor.py --name Team_07 --url PRACTICE_URL --token CLASS_TOKEN
```

FLAG	MEANING
`--name`	Required. Your team name, kept for the whole assignment.
`--url`	Required. The practice or tournament endpoint, ending in `/agent`.
`--token`	The class token.
`--machine`	Optional label shown on the leaderboard, for example `"M3 Pro, 12 cores"`.

The constructor takes two more arguments that the command line does not expose. `auto_calibrate=False` skips the startup benchmark, which is handy when you are testing bidding logic and do not want to wait. It leaves `self.rates` empty, so `self.estimate` returns infinity until you fill them in yourself. `verbose=False` silences the log.